Архитектура компьютеров и операционные системы

Лабораторная работа №2

true

Цель работы

Изучить идеологию и применение средств контроля версий и освоить умения по работе с git.

Задания

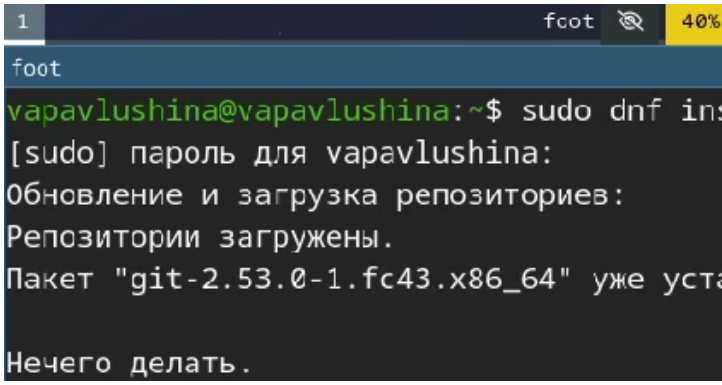
-Создать базовую конфигурацию для работы с git. -Создать ключ SSH. -Создать ключ PGP. -Настроить подписи git. -Зарегистрироваться на Github. -Создать локальный каталог для выполнения заданий по предмету.

Теоретическое введение

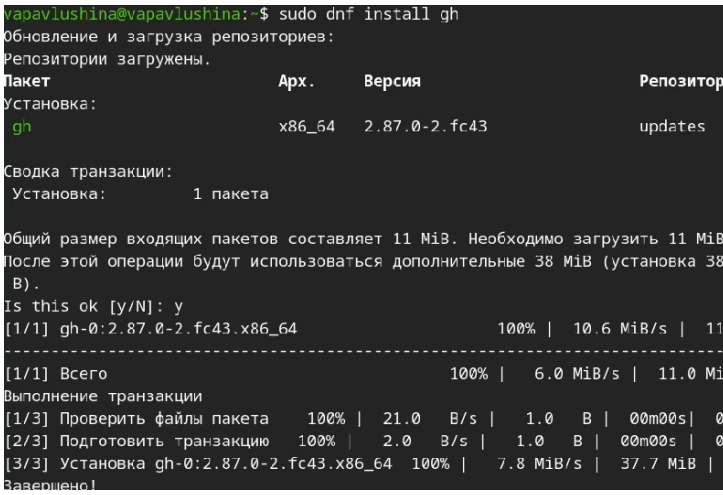
Системы контроля версий нужны для работы нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

Выполнение лабораторной работы

Установка программного обеспечения

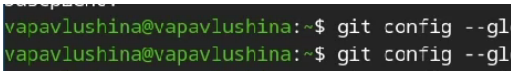


Устанавливаем git ([рис. @fig:lab02-1]). lab02-1}



Устанавливаем gh ([рис. @fig:lab02-2]). lab02-2}

Базовая настройка git



Задаём имя пользователя и email ([рис. @fig:lab02-3]). lab02-3}

Настраиваем utf-8 в выводе сообщений git ([рис. @fig:lab02-4]).

```
vapavlushina@vapavlushina:~$ git config --global core.quotepath false
```

{#fig: lab02-4}

Настроим верификацию и подписание коммитов git. Для этого зададим имя начальной ветки ([рис. @fig:lab02-5]) и настроим параметры autocrlf, safecrlf ([рис. @fig:lab02-6]).

```
vapavlushina@vapavlushina:~$ git config --global init.defaultBranch master
```

{#fig: lab02-5}

```
vapavlushina@vapavlushina:~$ git config --global core.autocrlf input
vapavlushina@vapavlushina:~$ git config --global core.safecrlf warn
```

{#fig: lab02-6}

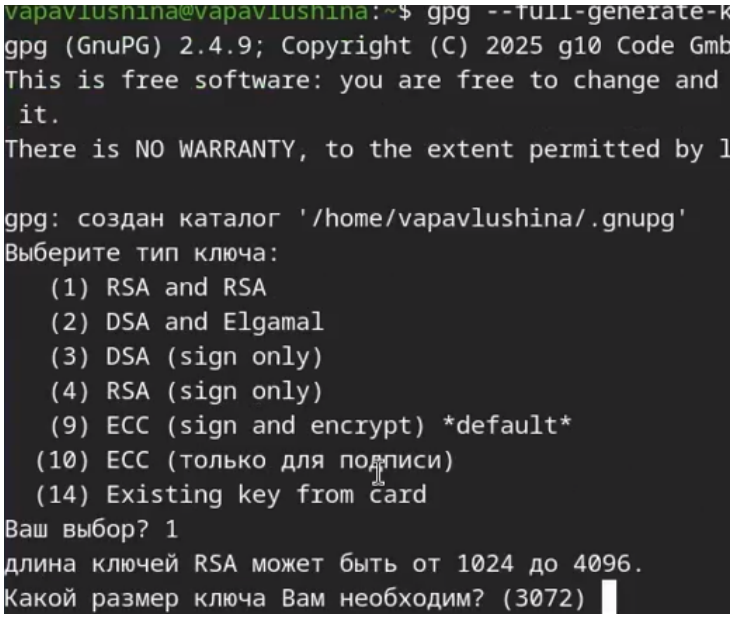
Создание ключа ssh

Создадим по алгоритму rsa с ключом размером 4096 бит ([рис. @fig:lab02-7]).

```
vapavlushina@vapavlushina:~$ ssh-keygen -t rsa -b 4096 -C "akutagawa_39@bk.ru"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/vapavlushina/.ssh/id_rsa):
Created directory '/home/vapavlushina/.ssh'.
Enter passphrase for "/home/vapavlushina/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/vapavlushina/.ssh/id_rsa
Your public key has been saved in /home/vapavlushina/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:ggB3m94qKWPlZtncVXhnaTOK98bNwTN9jw77+7TW8Q0 akutagawa_39@bk.ru
The key's randomart image is:
+---[RSA 4096]----+
|. . .            |
| o . o    .   .  |
|  . o    . o B   |
|   o o    + = + .|
|   .o o So o   =o|
|  o.+.... . o Eo*|
|o.o=.o .    .+.+B|
|.oo.        .+ o=|
|            ..=+.|
+----[SHA256]-----+
```

{#fig: lab02-7}

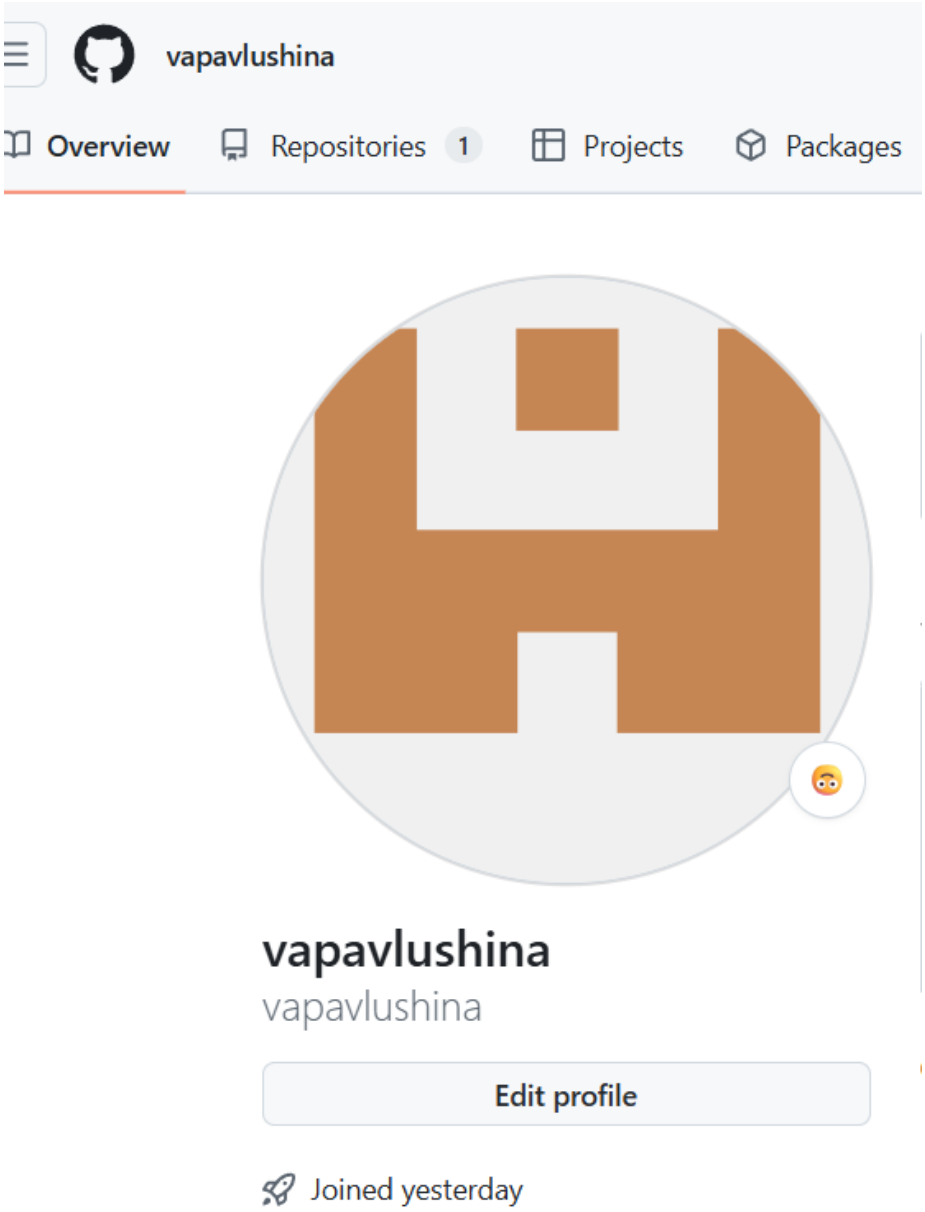
Создание ключа pgp



Генерируем ключ ([рис. @fig:lab02-8]). lab02-8}

Настройка github

Создали учётную запись на github ([рис. @fig:lab02-9]).



{#fig:lab02-9}

Добавление PGP ключа в GitHub

Выводим список ключей и копируем отпечаток приватного ключа ([рис. @fig:lab02-10], ([рис. @fig:lab02-11)).

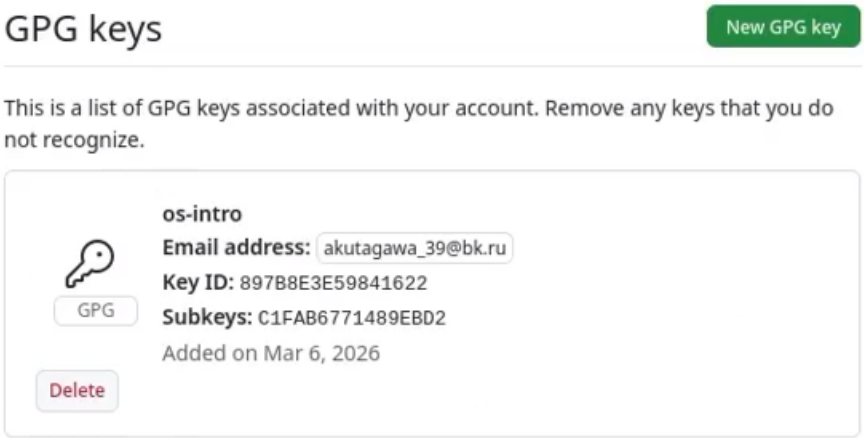
```
vapavlushina@vapavlushina:~$ gpg --list-secret-keys --keyid-
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model:
gpg: глубина: 0  достоверных:   1  подписанных:   0  доверие
1u
[keyboxd]
---------
sec   rsa4096/897B8E3E59841622 2026-03-05 [SC]
      4EBD6AAFDB9AFE4826FD17A3897B8E3E59841622
uid               [  абсолютно ] vapavlushina <akutagawa_39@
ssb   rsa4096/C1FAB6771489EBD2 2026-03-05 [E]
```

lab02-10}

```
vapavlushina@vapavlushina:~$ gpg --armor --export 897B8E3E59841622
```

{#fig: lab02-11}

Перейдём в настройки GitHub, нажмём на кнопку New GPG key и вставим полученный ключ в поле для ввода ([рис. @fig:lab02-12]).

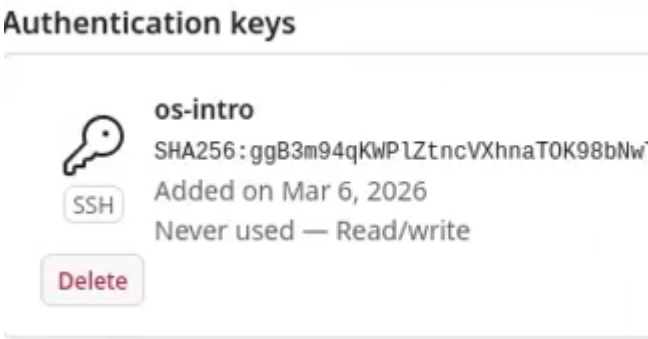


{#fig: lab02-12}

Добавление SSH ключа в GitHub

Для этого с помощью cat выедем отпечаток ключа SSH и скопируем его в буфер обмена ([рис. @fig:lab02-13]).

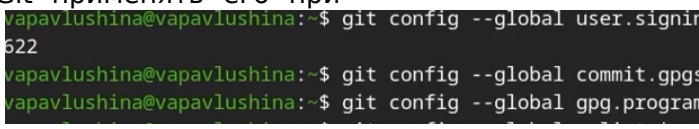
{#fig:lab02-13}



И вставим его в GitHub ([рис. @fig:lab02-14]).
lab02-14}

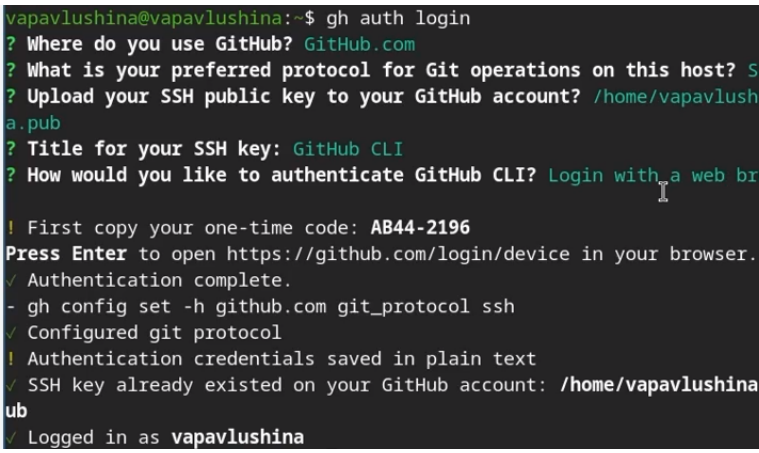
Настройка автоматических подписей коммитов git

Используя введённый email, укажем Git применять его при подписи коммитов ([рис. @fig:lab02-15]).



lab02-15}

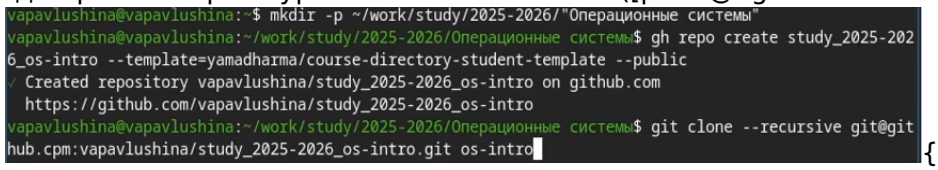
Настройка gh



Авторизуемся ([рис. @fig:lab02-16]). {#fig:lab02-16}

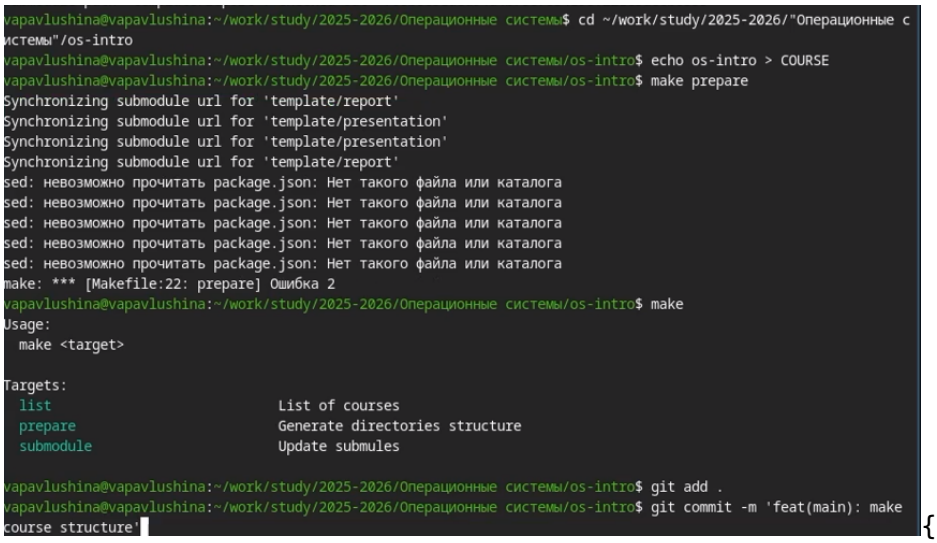
Шаблон для рабочего пространства

Создадим репозиторий курса на основе шаблона ([рис. @fig:lab02-17]).



{#fig:lab02-17}

Настраиваем каталог курса. Перейдём в каталог курса, удалим лишние файлы и создадим необходимые каталоги ([рис.



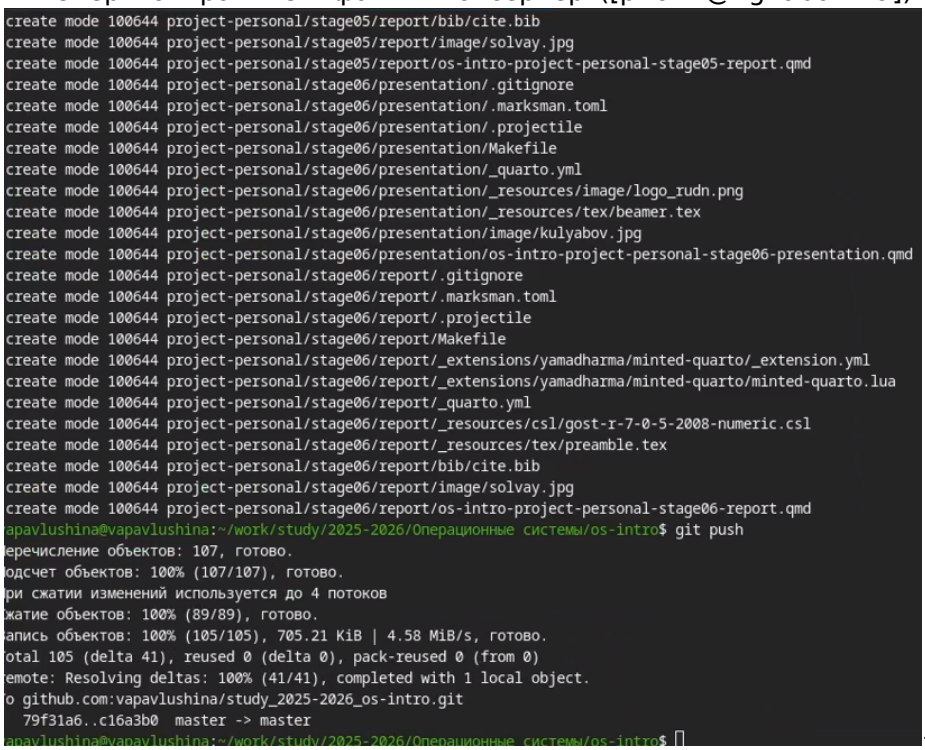
```
vapavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы$ cd ~/work/study/2025-2026/"Операционные с
истемы"/os-intro
vapavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$ echo os-intro > COURSE
vapavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$ make prepare
Synchronizing submodule url for 'template/report'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/report'
sed: невозможно прочитать package.json: Нет такого файла или каталога
sed: невозможно прочитать package.json: Нет такого файла или каталога
sed: невозможно прочитать package.json: Нет такого файла или каталога
sed: невозможно прочитать package.json: Нет такого файла или каталога
sed: невозможно прочитать package.json: Нет такого файла или каталога
make: *** [Makefile:22: prepare] Ошибка 2
vapavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$ make
Usage:
  make <target>

Targets:
  list                            List of courses
  prepare                         Generate directories structure
  submodule                       Update submules

vapavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$ git add .
vapavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -m 'feat(main): make
course structure'
```

@fig:lab02-18]). {#fig:lab02-18}

И теперь отправляем файлы на сервер ([рис. @fig:lab02-19]).



```
create mode 100644 project-personal/stage05/report/bib/cite.bib
create mode 100644 project-personal/stage05/report/image/solvay.jpg
create mode 100644 project-personal/stage05/report/os-intro-project-personal-stage05-report.qmd
create mode 100644 project-personal/stage06/presentation/.gitignore
create mode 100644 project-personal/stage06/presentation/.marksman.toml
create mode 100644 project-personal/stage06/presentation/.projectile
create mode 100644 project-personal/stage06/presentation/Makefile
create mode 100644 project-personal/stage06/presentation/_quarto.yml
create mode 100644 project-personal/stage06/presentation/_resources/image/logo_rudn.png
create mode 100644 project-personal/stage06/presentation/_resources/tex/beamer.tex
create mode 100644 project-personal/stage06/presentation/image/kulyabov.jpg
create mode 100644 project-personal/stage06/presentation/os-intro-project-personal-stage06-presentation.qmd
create mode 100644 project-personal/stage06/report/.gitignore
create mode 100644 project-personal/stage06/report/.marksman.toml
create mode 100644 project-personal/stage06/report/.projectile
create mode 100644 project-personal/stage06/report/Makefile
create mode 100644 project-personal/stage06/report/_extensions/yamadharma/minted-quarto/_extension.yml
create mode 100644 project-personal/stage06/report/_extensions/yamadharma/minted-quarto/minted-quarto.lua
create mode 100644 project-personal/stage06/report/_quarto.yml
create mode 100644 project-personal/stage06/report/_resources/csl/gost-r-7-0-5-2008-numeric.csl
create mode 100644 project-personal/stage06/report/_resources/tex/preamble.tex
create mode 100644 project-personal/stage06/report/bib/cite.bib
create mode 100644 project-personal/stage06/report/image/solvay.jpg
create mode 100644 project-personal/stage06/report/os-intro-project-personal-stage06-report.qmd
apavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$ git push
еречисление объектов: 107, готово.
одсчет объектов: 100% (107/107), готово.
ри сжатии изменений используется до 4 потоков
жатие объектов: 100% (89/89), готово.
апись объектов: 100% (105/105), 705.21 KiB | 4.58 MiB/s, готово.
otal 105 (delta 41), reused 0 (delta 0), pack-reused 0 (from 0)
emote: Resolving deltas: 100% (41/41), completed with 1 local object.
o github.com:vapavlushina/study_2025-2026_os-intro.git
  79f31a6..c16a3b0  master -> master
apavlushina@vapavlushina:~/work/study/2025-2026/Операционные системы/os-intro$
```

{#fig:lab02-19}

Выводы

Изучила идеологию и применение средств контроля версий и освоила умения по работе с git.

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются? - VCS - это программные инструменты, которые отслеживают изменения в файлах и позволяют управлять историей этих изменений. Предназначены для хранения истории проекта, отката к старым версиям, совместной работы над одним проектом.
2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия. - Хранилищем является репозиторий, он хранит в себе все файлы проекта и историю их изменений. Commit - это фиксация измений файлов в проекте. История - это журнал всех когда-либо сделанны коммитов, по которому можно перемещаться. Рабочая копия - это локальная версия файлов проекта, с которой пользователь работает в данный момент.
3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида. - Централизованные VCS: есть одно главное хранилище на сервере. Работать без сети нельзя. Примеры: CVS. Десцентрализованные: у каждого разработчика на компьютере есть полная копия всего хранилища. Работать можно без интернета. Примеры: Git.
4. Опишите действия с VCS при единоличной работе с хранилищем. - Создать локальное хранилище(git init), создавать/редактировать файлы в рабочей папке, добавлять файлы в область подготовленных (git add), фиксировать изменения с комментарием(git commit), при необходимости можно просматривать историю (git log).
5. Опишите порядок работы с общим хранилищем VCS. - Получить актуальную версию из общего репозитория(git pull), внести свои изменения, отправить свои изменения в общее хранилище(git push), при необходимости можно будет сделать слияние изменений и после снова их отправить.
6. Каковы основные задачи, решаемые инструментальным средством git? - Отслеживание истории изменений, обеспечение одновременной работы нескольких разработчиков,

управление ветками, синхронизация изменений между локальным и удалёнными репозиториями.

7. Назовите и дайте краткую характеристику командам git.
   - git init (создать новый репозиторий)
   - git clone (скопировать существующий репозиторий к себе)
   - git add (добавить файлы в индекс)
   - git commit (зафиксировать изменения)
   - git status (посмотреть состояние файлов)
   - git log (посмотреть историю коммитов)
   - git push (отправить коммиты в удалённый репозиторий)
   - git pull (скачать изменения из удалённого репозитория)
   - git branch (работа с ветками)
8. Приведите примеры использования при работе с локальным и удалённым репозиториями. - Локальный репозиторий: git init -> git add . -> git commit -m “comment” -> git log. Удалённый репозиторий: git clone -> git add . -> git commit -m “comment” -> git pull -> git push.
9. Что такое и зачем могут быть нужны ветви (branches). Ветви - это указатели на определённые коммиты, позволяющие вести параллельную разработку. Нужны, чтобы разрабатывать новые функции или исправлять ошибки изолированно.
10. Как и зачем можно игнорировать некоторые файлы при commit? - Нужно, чтобы случайно не закоммитить служебные или временные файлы (с паролями, кэш итд). Для этого создаётся файл .gitignore, в котором прописываются шаблоны имен файлов/каталогов, которые Git должен игнорировать.